SQL Quick Reference — SQLite Desk Guide

Basic Retrieval

Signal	Clause	Example
Everything	SELECT *	SELECT * FROM orders
Specific cols	SELECT c1, c2	SELECT id, order_date FROM orders
Sort	ORDER BY	ORDER BY order_date DESC
Limit rows	LIMIT	SELECT * FROM orders LIMIT 100
No duplicates	DISTINCT	SELECT DISTINCT customer_id FROM orders

Filtering

Signal	Clause	Example
Exact match	= / !=	WHERE status = 'shipped'
Multiple values	IN / NOT IN	WHERE region IN ('E','W')
Range	BETWEEN	WHERE order_date BETWEEN a AND b
Pattern	LIKE '%x%'	WHERE email LIKE '%@gmail%'
Missing	IS NULL	WHERE ship_date IS NULL
Present	IS NOT NULL	WHERE ship_date IS NOT NULL
Multi-condition	AND / OR / ()	WHERE (a OR b) AND c
Dirty data	TRIM(col)=''	WHERE TRIM(status)='' OR status IS NULL
NULL default	COALESCE(col, default)	COALESCE(status, 'Unknown')
Unmatched after LEFT JOIN	b.id IS NULL	WHERE b.id IS NULL

Aggregation & Grouping

'how many'→COUNT(*) · 'total'→SUM · 'average'→AVG · 'max/min'→MAX/MIN

Signal	Function	Example
Count rows	COUNT(*)	SELECT COUNT(*) FROM orders
Count non-null	COUNT(col)	COUNT(ship_date)
Count if	COUNT(CASE WHEN..THEN 1 END)	counts matches, ignores NULL
Sum if	SUM(CASE WHEN..THEN 1 ELSE 0 END)	same result, alt. syntax
Percent if	1.0*COUNT(CASE..)/ COUNT(*)	×100 for a percentage
Group	GROUP BY	SELECT region, COUNT(*) FROM orders GROUP BY region
Filter groups	HAVING	GROUP BY id HAVING COUNT(*) > 10
Join to string	GROUP_CONCAT	GROUP_CONCAT(status, ', ')

Conditional Logic

Signal	Function	Example
Categorize	CASE WHEN..THEN..ELSE..END	CASE WHEN status='A' THEN 'Active' ELSE 'Other' END
NULL default	COALESCE(col, default)	COALESCE(status, 'Unknown')
NULL if match	NULLIF(col, value)	NULLIF(region, '')

Date & Time

Dates stored as ISO text ('2026-07-11'); SQLite reads them natively.

```
date('now')                    -- today
strftime('%Y-%m', order_date)  -- year-month
CAST(julianday(a)-julianday(b) AS INTEGER)  -- days between
date(order_date, '+7 days')    -- add/subtract time
```

Dates: https://www.sqlitetutorial.net/sqlite-date/

String Manipulation

Signal	Function	Example
Replace text	REPLACE(col,'a','b')	REPLACE(code,'WH-','')
Trim spaces	TRIM(col)	TRIM(status)
Pattern end	LIKE '%x'	WHERE code LIKE '%EAST'
Case	UPPER / LOWER	UPPER(status)='ACTIVE'
Substring	SUBSTR(col,start,len)	SUBSTR(code,1,3)
Concatenate	col1 || col2	first || ' ' || last
Zero-pad	PRINTF('%02d', n)	PRINTF('P%02d', id)

|| returns NULL if either side is NULL; concat() treats NULL as empty string.

Strings: https://www.sqlitetutorial.net/sqlite-string-functions/sqlite-concat/

Joins

Type	Behavior	Example
INNER JOIN	Only matches in both	a JOIN b ON a.id=b.id
LEFT JOIN	All left rows; NULL if no match	a LEFT JOIN b ON a.id=b.id
RIGHT JOIN	All right rows; NULL if no match	a RIGHT JOIN b ON a.id=b.id
FULL OUTER	All rows both sides	a FULL OUTER JOIN b ON a.id=b.id

RIGHT/FULL OUTER JOIN need SQLite ≥3.39 — else rewrite as LEFT JOIN, tables swapped.

Subqueries

Placement	Use when…	Example
WHERE	vs. row-level value	WHERE price=(SELECT MAX(price) FROM t)
HAVING	vs. aggregated value	HAVING AVG(x)>(SELECT AVG(x) FROM t)
FROM	derived table first	FROM (SELECT..) WHERE recency=1
JOIN..ON	dedupe before joining	JOIN (SELECT DISTINCT..) s ON..

```
WITH cte AS (SELECT..) SELECT AVG(x) FROM cte; -- CTE = cleaner nested subquery
```

Window Functions

Signal	Function	Example
Most recent per group	ROW_NUMBER() OVER(PARTITION.. ORDER..DESC)	recency=1 → latest
No collapsing	COUNT(*) OVER(PARTITION BY col)	group size per row
Running total	SUM(col) OVER(ORDER BY col)	cumulative sum
Rank	RANK() OVER(ORDER BY col DESC)	1=highest
Prior row	LAG(col) OVER(ORDER BY col)	vs. previous period

Combining Results

Signal	Clause	Behavior
Stack, no dupes	UNION	removes duplicate rows
Stack, keep all	UNION ALL	keeps duplicates
In both sets	INTERSECT	rows present in both
In first, not second	EXCEPT	rows only in first query

Clause Execution Order

Written: SELECT·FROM·JOIN·WHERE·GROUP BY·HAVING·ORDER BY·LIMIT

Executed: FROM·JOIN·WHERE·GROUP BY·HAVING·SELECT·ORDER BY·LIMIT

WHERE runs before SELECT — no filtering on a column alias; use HAVING or a subquery.